

# MEMLLM: Finetuning LLMs to Use An Explicit Read-Write Memory

Ali Modarressi<sup>1,2</sup> Abdullatif Köksal<sup>1,2</sup> Ayyoob Imani<sup>1,2</sup>

Mohsen Fayyaz<sup>3</sup> Hinrich Schütze<sup>1,2</sup>

<sup>1</sup>Center for Information and Language Processing, LMU Munich, Germany

<sup>2</sup>Munich Center for Machine Learning, Germany <sup>3</sup>Microsoft, Berlin, Germany

amodaresi@cis.lmu.de

## Abstract

While current large language models (LLMs) demonstrate some capabilities in knowledge-intensive tasks, they are limited by relying on their parameters as an implicit storage mechanism. As a result, they struggle with infrequent knowledge and temporal degradation. In addition, the uninterpretable nature of parametric memorization makes it challenging to understand and prevent hallucination. Parametric memory pools and model editing are only partial solutions. Retrieval Augmented Generation (RAG) – though non-parametric – has its own limitations: it lacks structure, complicates interpretability and makes it hard to effectively manage stored knowledge. In this paper, we introduce MEMLLM, a novel method of enhancing LLMs by integrating a structured and explicit read-and-write memory module. MEMLLM tackles the aforementioned challenges by enabling dynamic interaction with the memory and improving the LLM’s capabilities in using stored knowledge. Our experiments indicate that MEMLLM enhances the LLM’s performance and interpretability, in language modeling in general and knowledge-intensive tasks in particular. We see MEMLLM as an important step towards making LLMs more grounded and factual through memory augmentation.

## 1 Introduction

State-of-the-art large language models (LLMs) perform well in knowledge-intensive tasks (Yu et al., 2023; Chowdhery et al., 2023). They solve these tasks utilizing the information memorized in their vast array of parameters (Roberts et al., 2020). However, the effectiveness of parameter-based memorization is limited for infrequent entities and concepts (Kandpal et al., 2023; Mallen et al., 2023) and is prone to temporal degradation (Kasai et al., 2023; Jang et al., 2022). Model editing may address some of these issues (Sinitsin et al., 2020; De Cao et al., 2021; Mitchell et al., 2022), but

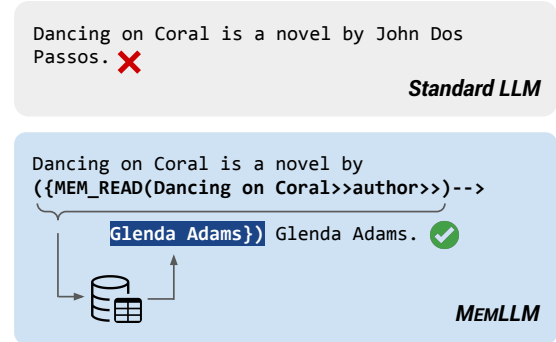


Figure 1: A vanilla pretrained LLM can produce hallucinated nonfactual output. In contrast, MEMLLM dynamically queries its explicit memory for stored facts, resulting in more factually grounded text generation.

it struggles with maintaining locality – possibly damaging model performance in unrelated areas (Yao et al., 2023b; Gu et al., 2024). Augmenting LLMs with extra parameters such as memory pools can preserve knowledge for subsequent utilization (Wang et al., 2023a, 2024). However, parametric memorization is prone to distortion and can lead to hallucinated nonfactual output. In addition, parametric mechanisms such as memory pools have limited capacity and lack interpretability (Maynez et al., 2020; Ji et al., 2023).

Alternatively, Retrieval Augmented Generation (RAG) integrates non-parametric knowledge by making external knowledge available to the model via retrieved text chunks (Gua et al., 2020; Lewis et al., 2020). The stored knowledge in RAG is inherently unstructured. This has two disadvantages. (1) Storing, indexing and (during inference) retrieving the knowledge is inefficient (Gao et al., 2023; Liu et al., 2023; Levy et al., 2024). (2) Even though it is non-parametric, managing and editing the knowledge is challenging due to RAG’s lack of structure.

Another approach is to augment LLMs with a non-parametric memory component that inter-

acts with the LLM either through natural language text or a formalized API (Wang et al., 2023b). Although prior work has demonstrated enhanced abilities in extended dialogs, long-text generation and question-answering (Packer et al., 2023; Hu et al., 2023; Zhou et al., 2023), these methods are primarily prompt-dependent and necessitate customization for each specific task and model. They also suffer from the lack of a structured memory. This undermines interpretability and interoperability (Wang et al., 2023b).

In this paper, we introduce MEMLLM, an LLM endowed with an explicit memory component. It has the general advantages of some of the previous memory-focused work we discussed: we can **keep information accessible indefinitely**, beyond the context window, including **infrequent information** that standard LLMs struggle with.

The LLM has **both read and write access** to the memory component, i.e., it can store information in the memory as it processes text (or interacts with a user) and retrieve it when it needs it (Figure 1). We specify an **API for read and write access**. The LLM issues read commands to retrieve from the memory and write commands to write to the memory.

We adopt **finetuning to train the model** to access the memory. Based on the API specification, we create a dataset with training examples of API read and write commands and finetune the LLM on it. Our published training dataset can be used to finetune any language model, endowing it with explicit memory without requiring architectural changes.

Our memory has an **explicit structured** schema, similar to a database schema. It is **interpretable** and **inspectable** for humans. It is **editable** by humans. It is **scalable** since databases have excellent scalability properties. It is **interoperable** since the contents of the memory can be exported (e.g., to a different LLM supporting explicit memory) and contents can be imported from data resources (e.g., from Wikidata).

Our evaluation on DOCCRED (Yao et al., 2019) demonstrates that MEMLLM achieves better perplexity compared to baselines without memory components, with strong gains on named entities.

## 2 Related Work

Work on augmenting large language models (LLMs) with memory can be broadly categorized into the following three lines of work.

We refer to approaches that extend LLM architectures with additional trainable parameters that are intended to store information long-term as **parametric memory**.

**Retrieval augmented generation (RAG)** retrieves text snippets from a large document database and makes the retrieval results available in the LLM’s context window.

**External memories** store facts and other knowledge that the LLM can then access through a variety of mechanisms, thereby enhancing its ability to process and understand larger contexts and keeping a reliable record of knowledge it encountered in the more distant past.

### 2.1 Parametric Memory

The term *memory* is sometimes associated with recurrent architectures (Hochreiter and Schmidhuber, 1997; Cho et al., 2014) that represent past context with vectors. A similar mechanism is employed in transformer-based models that have been extended with memory tokens to process long segments of text. These tokens’ function is comparable to state vectors in RNNs, transferring context from previous segments to subsequent ones (Burtsev et al., 2020; Bulatov et al., 2022). An alternative is a *memory pool* (accessible to all segments), a large set of parameters that memorizes and shares information between multiple contexts (Wang et al., 2024).

While other recent advances use similar techniques with vector- or parameter-based memory systems that handle long-range dependencies and incorporate prior context (Martins et al., 2022; Wu et al., 2022a,b; Cheng et al., 2023; Wang et al., 2023a; He et al., 2024), they are constrained by the limited capacity of memory vectors (Jelassi et al., 2024). In contrast, MEMLLM has no architectural limitations on the number of relationships it can store. Equally important, because our memory is explicit, it is interpretable and editable.

### 2.2 Retrieval Augmented Generation

**Retrieval Augmented Generation (RAG)** augments the LLM’s input with relevant retrieved text snippets, often to induce factuality in the generated output (Gua et al., 2020; Lewis et al., 2020). Standard RAG retrieves documents based on the input prompt. The resulting extensive use of the context window often incorporates passages that are unnecessary or off-topic, leading to poor output quality (Shi et al., 2023). Some recent work pro-

posed improvements either by modifying the query (Jiang et al., 2023b) or by allowing the model to adopt a more dynamic approach in selecting and filtering relevant documents through a multistep process (Asai et al., 2023). While this enhances recall, the challenge of processing long documents remains. Other recent work improves the management of long context inputs (Beltagy et al., 2020; Wang et al., 2020), e.g., by utilizing the parametric memory techniques discussed above. However, even if the relevant documents are retrieved, this does not guarantee that the model can effectively use them due to the long context (Liu et al., 2023; Levy et al., 2024). Since our approach condenses the information into triples, it can effectively use a small context window.

### 2.3 External Memory

External memories provide storage that enhances the LLM’s ability to process and understand larger contexts and to keep a reliable record of facts and of knowledge in general. The external memory can be a database, a knowledge base, a knowledge graph and a Wikipedia (Guu et al., 2020; Lewis et al., 2020; Liu et al., 2022; Yao et al., 2023a), inter alia. The interactions with the external memory can be conducted in the form of natural (Park et al., 2023; Zhou et al., 2023) or formal language, such as using standardized APIs that facilitate the parsing of commands given by the model (Schick et al., 2023; Hu et al., 2023).

A recent trend in this line of work retains certain texts beyond the context window. This is commonly achieved by storing summarized information from previous contexts in an external memory for future retrieval (Park et al., 2023). A system prompt is crafted to delineate the input format and the methods for generating and utilizing the memory content. Leveraging task-specific prompts, this can enhance performance in diverse applications, including long-form generation, summarization, question answering and dialog coherence (Zhou et al., 2023; Packer et al., 2023; Chen et al., 2023; Liang et al., 2023).<sup>1</sup> MEMLLM can be seen as part of this category, but it features a structured format for storing information. While some other work also supports structured information storage

<sup>1</sup>OpenAI has not published technical details about their recent memory features (<https://openai.com/blog/memory-and-new-controls-for-chatgpt>). Nonetheless, from the demonstrations observed, it seems plausible that it employs a similar strategy by summarizing and storing sentences in memory for subsequent retrieval.

formats, its functionality is constrained to specific tasks, such as data record management (Hu et al., 2023). In addition, a notable limitation of this related work – mentioned earlier – is its dependence on the system prompt and the extent to which the language model can effectively utilize it. For example, in MemGPT (Packer et al., 2023), a different system prompt (each approximately 1024 tokens long) is provided for each task. This means that each new task necessitates the composition and evaluation of a new prompt. In contrast, we train our approach for generic language modeling to make it easily extensible to many different tasks.

The approach proposed by Modarressi et al. (2023) is broadly similar to ours, but they train and evaluate on synthetic data. In contrast, we construct our training data from a dataset of non-synthetic documents that have been annotated with relations on a finegrained level. Since we release our training data, they can be used for adding an explicit memory component to any trainable language model without requiring architectural changes.

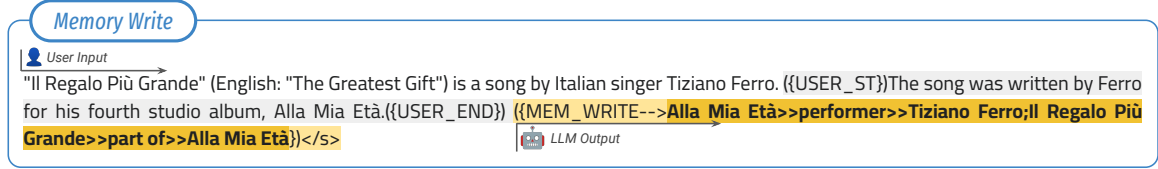
## 3 Methodology

So far we have outlined our motivation for the integration of an explicit memory into LLMs, including interpretability and editability. Now we turn to our approach to putting this integration into practice: finetuning using the standard language modeling objective. In this section, we present a finetuning regime that teaches the LLM (1) to extract knowledge from text and write it to the memory and (2) to read knowledge from the memory and leverage it for better language modeling.

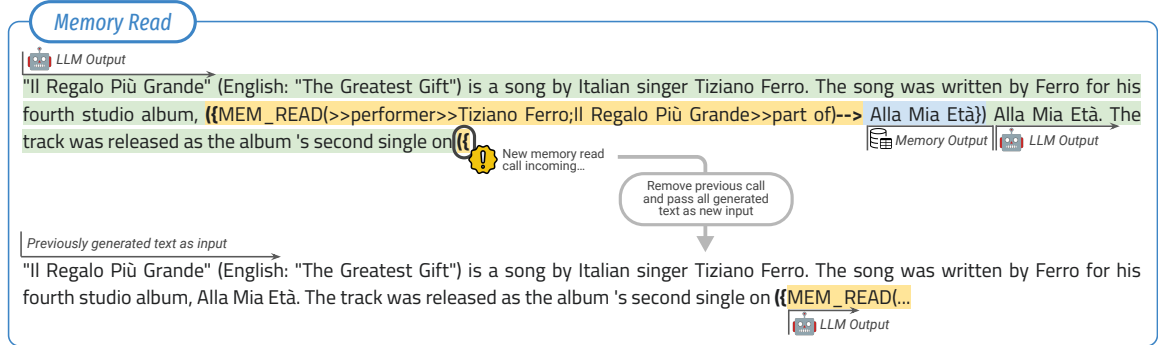
Following Schick et al. (2023) and Modarressi et al. (2023), we define an API to access the explicit memory. The API specifies how the LLM initiates memory writes and memory reads. We now describe the data structure of the explicit memory, the API and how we finetune the LLM to learn the API including how we create the training data for finetuning.

### 3.1 Memory Structure

The memory stores information in relation triples. Each relation is defined in the following format:  $r = \langle e_s, t, e_o \rangle$ , where  $e_s$  is the first entity or subject,  $e_o$  the second entity or object and  $t$  is the relation. Example:  $\langle \text{Washington D.C., capital of, United States} \rangle$ . The entities and relations are stored as raw text and



(a) For memory writes, the input is given in two parts. (i) The pretext provides context for the model (e.g., antecedents for pronouns). (ii) The focus sentence is the span of text from which the model is tasked to extract all relations that occur in that span. (`{{USER_ST}}`) and (`{{USER_END}}`) are tags marking the focus sentence, prompting the model to enter memory-write mode. Upon receiving the input, the model produces an API call starting with the (`{{MEM_WRITE-->` command followed by the **extracted relations**. The process concludes by closing the API call with an end-of-sentence token `}}</s>`. To extract relations from a full document, MEMLLM scans the document one sentence at a time.



(b) The model decodes one token at a time, as in standard language modeling. However, it is also trained to generate memory read commands whenever it predicts that a memory read would retrieve useful information at this point in the text. In the example, after decoding some tokens, the model generates a (`{{MEM_READ(>` command followed by **queries**. The `-->` token triggers execution of the queries. **Returned results** are appended to the text and the API call is closed. **The model can use the retrieved results for a more accurate decoding** for the continuation. The model continues regular language model decoding until it decides to invoke another memory read by generating the (`{{` token. Before proceeding with this new API call, we remove the previous one because it is unlikely to still be useful and in order to keep the input compact.

Figure 2: A visualization of inference with memory read and memory write

vectors, each in separate tables. **The vector representations abstract away** different surface forms that refer to the same entity, e.g., US, USA, United States, United State of America. We utilize Contriever (Izacard et al., 2022) as an off-the-shelf representation generator to create the vector representations. Intuitively, given that the **surface forms** of entities are of a phrasal-level size, recall and precision for entity queries is expected to be higher than in document-level retrieval.<sup>2</sup> In what follows, in the interest of brevity, we use the symbols  $e$  and  $t$  both for the entity/relation itself as well as for their vectors.

**Handling Queries** To query the memory, we define the following query format:

$$\mathbf{q} \in \{ \langle e_s^q, t^q, * \rangle, \langle *, t^q, e_o^q \rangle \}$$

<sup>2</sup>While typically the count of unique entities is significantly lower than the number of relations, the quantity of entities can still reach up to millions if the framework encounters a vast corpus of documents. Hence, we employ Hierarchical Navigable Small World graphs (HNSW) (Malkov and Yashunin, 2018) to facilitate rapid indexing for an extensive number of entities.

These two query templates give us sufficiently specific queries (as opposed to queries with two variables, for example) that are likely to return useful entity information.  $e_s^q$ ,  $e_o^q$  and  $t^q$  are subject entity, object entity and relation, respectively.  $*$  indicates the position in the triple of the entity we are querying for.

**For retrieval**, we first determine a set of candidate entities:

$$\mathcal{C} = \{e' | \cos(e^q, e') \geq \tau_e\}$$

That is, all entities with an above threshold similarity are considered candidate entities.

Similarly, we determine a set of candidate relations:

$$\mathcal{T} = \{t' | \cos(t^q, t') \geq \tau_t\}$$

If the query is a query for an object, i.e.,  $\mathbf{q} = \langle e_s^q, t^q, * \rangle$ , then we retrieve the following final set of entities from the memory:

$$\mathcal{E} = \{e_o | \exists (e, t, e_o) \in \mathcal{M} : e \in \mathcal{C}, t \in \mathcal{T}, 0.5(\cos(e, e_s^q) + \cos(t, t^q)) \geq \tau_r\}$$

where  $\mathcal{M}$  is the memory. That is, we look for all triples in the memory with entities/reactions from the candidate sets such that their average similarity to query subject and relation is above the threshold. Subject queries are handled analogously.

### 3.2 Memory-API & Inference

We now describe the API that specifies how the LLM initiates memory writes and memory reads.

#### 3.2.1 Memory writes

**For memory writes**, we process the input sentence by sentence. For sentence  $i$  the input  $x_i^{\text{MW}}$  to the LLM is formatted as follows:

$$x_i^{\text{MW}} = S_{<i} + (\{\text{USER\_ST}\}) + s_i + (\{\text{USER\_END}\})$$

where  $S_{<i}$  are the  $i - 1$  preceding sentences and  $s_i$  is the  $i$ -th sentence, bracketed by tags to mark it as the *focus sentence*.<sup>3</sup> The LLM’s task is then to extract all relations occurring in the focus sentence and to generate a write command that stores them in the memory:

$$y[x_i^{\text{MW}}] = (\{\text{MEM\_WRITE} \rightarrow e_s^1 \triangleright t^1 \triangleright e_o^1; e_s^2 \triangleright t^2 \triangleright e_o^2; \dots\})$$

Note that the context  $S_{<i}$  is in general necessary to correctly extract relations from the focus sentence, e.g., if the focus sentence refers to a previously introduced entity with a pronoun. We fine-tune the LLM to only extract relations from the focus sentence (not from the preceding context); see §3.3 for details.

To extract all relations from a document and write them to the memory, we iterate over the sentences of a document one by one.<sup>4</sup>

#### 3.2.2 Memory reads

**The LLM can at each point in time** either emit a regular token or initiate an API MEM\_READ call by generating:

$$(\{\text{MEM\_READ}(\$$

It then continues by generating one or more queries, subject or object queries as introduced above:  $\mathbf{q} \in \{ \langle e_s^q, t^q, * \rangle, \langle *, t^q, e_o^q \rangle \}$ . We specify the following syntax for the memory-read API call:

$$(\{\text{MEM\_READ}(e_s^{q1} \triangleright t^{q1} \triangleright e_o^{q2}; \dots) \rightarrow$$

<sup>3</sup>For sentence splitting, we use NLTK (Bird and Loper, 2004).

<sup>4</sup>We decode with a late stopping technique, not with greedy decoding. See Appendix A.

We then retrieve the entity sets  $\mathcal{E}$  from the memory as described in §3.1, merge them and append them to the API call:

$$(\{\text{MEM\_READ}(e_s^{q1} \triangleright t^{q1} \triangleright e_o^{q2}; \dots) \rightarrow e_1, e_2, e_3, \dots\})$$

The LLM then continues decoding. Figure 2b gives an example. The LLM starts generating a sentence that refers to an album by the Italian singer Tiziano Ferro. It has learned that just before naming the album is a good point at which to initiate a memory read. Two queries are generated (including: “What is the song “Il Regalo Più Grande” part of?”). One entity is returned by the memory (“Alla Mia Età”) and written to the buffer. The LLM then generates the name of the correct album (“Alla Mia Età”). **This example illustrates** that our memory has the potential of reducing hallucinations because an explicit representation of the knowledge that “Il Regalo Più Grande” is part of the album “Alla Mia Età” is available through the memory.

**We remove memory-read API calls** from the context if or once they are no longer useful. This happens in three cases. (i) The returned set  $\mathcal{E}$  of entities is empty.<sup>5</sup> (ii) The number of retrieved entities exceeds a threshold  $Q_{thr}$  ( $Q_{thr} = 30$ ). It is difficult for the model to identify the correct entity from a large set, so large retrieval results are unlikely to be helpful. (iii) The model initiates a new memory-read API call by emitting “{”.

**Our motivation for removal is that an API call** that is not (or no longer) useful may distract the LLM and should therefore be removed. Omitting API-specific verbiage preserves the text’s natural flow and reduces the context to those parts of the input that are still informative for high-quality generation.

### 3.3 Finetuning the LLM

**In this section**, we describe how we create the dataset for training the model to generate memory-write and memory-read API calls. One innovation of our work is that we create the API training data from DOCCRED, a resource consisting of a corpus annotated with entities and relations (Yao et al., 2019).

#### 3.3.1 Memory write data

As illustrated in Figure 3, we use the DOCCRED (Yao et al., 2019) documents and their annotated

<sup>5</sup>We continue with the second highest scoring token instead of  $(\{\text{MEM\_READ}(\$  if this happens.

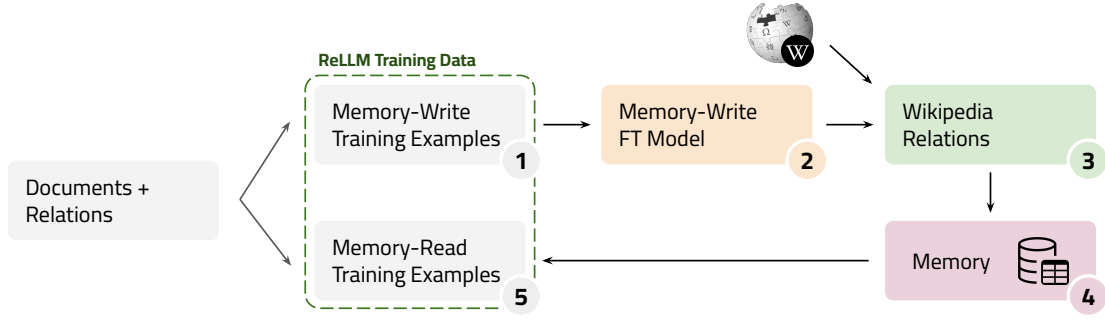


Figure 3: MEMLLM training data pipeline.

relations to construct memory-write training examples, subsequently fine-tuning a model with these examples.

In more detail, for each sentence  $s_i$  in a document, we retrieve from DOCRED all relation triples such that one entity has a full mention (i.e., not a pronoun) in  $s_i$  and the second entity has a full mention in either  $s_i$  or in one of the preceding sentences ( $S_{<i}$ ). We then structure the memory-write training example for  $s_i$  as outlined in §3.2.1: it consists of the context  $x_i^{\text{MW}}$  and the memory-write command  $y[x_i^{\text{MW}}]$ . Since the goal of this step is to teach the LLM to generate memory-write API calls, we compute the training loss on  $y[x_i^{\text{MW}}]$  only.

Note that the set of relation triples can be empty for a sentence  $s_i$ . In that case we generate a memory-write command in  $y[x_i^{\text{MW}}]$  that contains no relations. This encourages the LLM not to generate spurious relations for such “empty” sentences.

### 3.3.2 Memory-Read Data

For effective memory reads, the LLM has to learn (i) to identify where to initiate a query to the memory, (ii) to generate queries that retrieve helpful information from the memory and (iii) to make good use of the information that is returned by the memory. In this section, we describe how we generate our training data with the goal of teaching the LLM all three capabilities.

Given a DOCRED document  $d$ , we generate a different training instance  $d'$  for each memory-read API call. To produce  $d'$ , we scan  $d$ 's annotated entity mentions from the beginning to the end of the document. For each entity mention  $e_{\text{target}}$ , we collect all relation triples in which it participates. Such triples are a good basis for memory-read API calls that – when issued before  $e_{\text{target}}$  first appears – will help the LLM to correctly generate  $e_{\text{target}}$ ; this is why we refer to  $e_{\text{target}}$  as the target entity. We keep only that subset of the triples in which the mention

of the other entity  $e_q$  that the triple refers to (the query entity) has already occurred. (The LLM will in general not be able to generate a query containing  $e_q$  if  $e_q$  has not yet occurred.) We also discard all triples that we previously encountered during our scan. (These are already known at this point, so there is little utility initiating a query for them.) We then generate a query for each remaining triple: either  $\langle e_q, t, * \rangle$  (if  $e_{\text{target}}$  is the object of the triple) or  $\langle *, t, e_q \rangle$  (if  $e_{\text{target}}$  is the subject of the triple). The memory-read API call for the queries generated for  $e_{\text{target}}$  is placed immediately preceding  $e_{\text{target}}$ . This will retrieve  $e_{\text{target}}$  from the memory in many cases and then make it easy for the LLM to correctly predict  $e_{\text{target}}$  at the next position.

Next we retrieve results for the query from the memory. The memory we use here is the one that is populated from Wikipedia by the trained memory-write model as described in §3.3 and Figure 3. This simulates our intended scenario of longterm and large scale use of the memory. The memory write model misses some relations and incorrectly identifies others, resulting in an imperfect memory. We intentionally use this imperfect memory because it closely aligns the training data with the ultimate inference conditions.

If the query returns a large number of results from memory (more than  $Q_{\text{thr}} = 30$ ), we discard it as unlikely to be helpful. (See Appendix B for details.) An example is the query  $\langle *, \text{country}, \text{United States} \rangle$  where Wikidata defines the relation “country” as “sovereign state that this item is in”. There are thousands of entities that satisfy this query. Such an unspecific result does not provide any useful information to the LLM. For all other queries, we add the queries and the query result to the training document, following the format given in §3.2.2. See Figure 2b for an example.

Finally, we add the rest of  $d$  to the training document  $d'$  up to the next initiation of a memory read (indicated by “(”) or (if there is no next memory read) the entire remainder of  $d$ .<sup>6</sup>

To summarize, each training example  $d'$  is a concatenation of (i) the pretext, including the first two letters of the API call, i.e., “(”, (ii) the API call proper “MEM\_READ( $e_s^{q1} \gg t^{q1} \gg \dots$ )-->”, (iii) the query results returned from the memory “ $e_1, e_2, e_3, \dots$  }” and (iv) the posttext. The posttext either consists of the rest of the document if there is another memory read; in that case the posttext ends with “(”. Or (if there are no more memory reads) it consists of the entire rest of the document.

The loss is applied to the API call (ii) – this teaches the model to generate the correct API call. The loss is also applied to the posttext – this teaches the model to make good use of the information provided in the query results for predicting entities in the text; it also teaches the model to predict the next memory read (as indicated by “(”). (iii) is not subject to the loss since the query results are generated by the memory, not by the LLM. For the training example  $d'$  that contains the very first “(MEM\_READ(” in the document (and only for this  $d'$ ), the loss is also applied to the pretext – because the LLM needs to learn where to generate this first “(MEM\_READ(”.

## 4 Experiments

To train and evaluate MEMLLM, we construct training and evaluation datasets as described in Section 3.3. The essential requirement for this task is a dataset in which documents are annotated with extractable entities and relations. To this end, we use DOCRED, composed of Wikipedia texts annotated with named entity mentions, coreference information and intra- and inter-sentence relations (Yao et al., 2019). The relations are annotated in a Wikidata format; there are 96 different relations. In addition to validation and test sets, DOCRED has two training sets: one human-annotated, one created by distant supervision. The human-annotated set offers explicit evidence annotations for relations, similar to the validation set. In contrast, the distantly supervised set, which is crucial for us

due to its much larger size, lacks evidence annotations because of its automated annotation method. Therefore, this set includes false positives, such as relations that are not evident from the text or are misplaced. To address this, we implement a few-shot-based filtering approach to detect and remove false-positive relations. The resulting cleaned up distantly supervised data are then combined with the human-annotated data. See Appendix D for details.

Our evaluation measure is perplexity. Following Liu et al. (2022), we report (1) OVERALL PPL (PPL on the entire input text), (2) TARGET PPL (PPL on the target entities) and (3) ENTITY PPL (PPL on all named entities).<sup>7</sup>

We finetune two Mistral-7B (Jiang et al., 2023a) models using LoRA (Hu et al., 2022), a memory-write model and a memory-read model. See Appendix E for details on fine-tuning and memory retrieval hyperparameters. Our baselines are the original Mistral-7B and the memory-read model with its memory capabilities disabled. This latter memory-read baseline allows us to ascertain to what extent improvements are due to in-domain finetuning (as opposed to the explicit memory).

### 4.1 Results

Table 1 gives perplexity results on the DOCRED validation set. MEMLLM outperforms the two baselines on all three PPL measures (①). TARGET PPL improves by more than 0.1. This increase, for relations appearing for the first time in the text, suggests that memory reads successfully recall relevant information for language modeling. This improvement benefits not just all entities in the text (ENTITY PPL) but the entire text (OVERALL PPL) as well. The improvement using MEMLLM on target entities, which is the focus of this work, is around 15% while in-domain improvement due to finetuning is only 2.5%. This shows the effectiveness of MEMLLM when it comes to target entities, which is essential to generate factual text and prevent hallucination.

**Memory-Read Analysis.** For this analysis, instead of using the LLM to write to the memory, we directly populate the memory with the relations from the validation set (indicated as “DOCRED

<sup>6</sup>In the experiments, we observe that some calls are out of place or early (i.e., not immediately before the target entity). Therefore, we add random shifts in the calls to train a more robust model that can handle these early calls. See Appendix C for details.

<sup>7</sup>In a standard evaluation setup, PPL for an input text is computed in parallel, i.e., it can be computed in one forward pass. We instead compute PPL token by token since memory calls are produced mid-generation and subsequent generation is conditioned on them, an inherently sequential process.

|                             | Memory               | OVERALL | PPL<br>TARGET | ENTITY |
|-----------------------------|----------------------|---------|---------------|--------|
| Baseline #1 (Mistral-7b)    |                      | 1.774   | 1.180         | 1.420  |
| Baseline #2 (Mem. Disabled) | —                    | 1.616   | 1.151         | 1.355  |
| ① MEMLLM                    | MW[Wikipedia (Full)] | 1.606   | 1.009         | 1.322  |
| ② MEMLLM                    | MW[Wikipedia (Abs.)] | 1.607   | 1.013         | 1.324  |
| ③ MEMLLM                    | MW[DOCRED (Val.)]    | 1.589   | 0.945         | 1.289  |
| ④ + Gold MR Pos. & Queries  |                      | 1.578   | 0.786         | 1.287  |
| ⑤ MEMLLM                    | DOCRED (Val.)        | 1.589   | 0.913         | 1.290  |
| ⑥ + Gold MR Position        |                      | 1.579   | 0.847         | 1.268  |
| ⑦ + Gold Queries            |                      | 1.562   | 0.619         | 1.202  |
| ⑧ + Gold Target             |                      | 1.544   | 0.448         | 1.152  |
| ⑨ + Gold Target Only        |                      | 1.535   | 0.337         | 1.124  |

Table 1: MEMLLM performance on OVERALL PPL (on the entire text), TARGET PPL (target entities) and ENTITY PPL (all entities). We show the effect of changing the memory content to other sources. Memories labeled as "MW[X]" are populated with relations generated through memory-writes using documents from source X. For ⑤–⑨, the relations are directly from the validation set of DOCRED.

(Val.)” in column “Memory”). This lets us investigate what would happen if the memory-write process were error-free, i.e., all remaining errors are due to the memory-read process. We look at four potential sources of error in memory reads and present in each case an ablation in which this source of error is eliminated: the position of the memory read is the gold position immediately before the target entity (⑥ “Gold MR Position”), the query to the memory is the gold query (⑦ “Gold Queries”), the correct target entity is returned by the memory (⑧ “Gold Target”), the correct target entity is returned by the memory and no other entities (⑨ “Gold Target Only”). ⑨ is the lower bound perplexity for perfect memory reads (and perfect memory writes).

Comparing ⑨ on the TARGET PPL with ⑧ (0.337 vs 0.448), shows the effect of “ambiguity”. This +0.111 gap in PPL is induced because the memory can return other targets in addition to the gold target.

Moving from ⑧ to ⑦ (0.448 to 0.619) indicates the impact of the memory retrieval process. In ⑦, we exclusively use the gold queries, but without ensuring the inclusion of the gold target in the results. Since some queries are too general they could exceed  $Q_{thr}$  (as discussed in Section 3.2.2). Therefore, the memory reads are omitted (due to the empty memory output) in this case in contrast to ⑧ where it gets the target result as its only result.

The next comparison highlights the impact observed when the LLM itself generates queries (⑥)

vs. when only gold queries are created (⑦). Finally, the effect of the model-determined position of the memory read versus the gold position (⑥ vs ⑤) is shown in the rise from 0.847 to 0.913.

**Memory-Write Performance.** To analyze the impact of memory writes on perplexity, we isolate this factor by using fixed gold memory-read positions and queries and a fixed input corpus for extracting relations (DOCRED validation set), varying only the method by which the memory is populated: by running the memory-write model on the input corpus (④) vs. by reading out the relations from the gold data and directly storing them in the memory (⑦). As expected, PPL improves when directly stored (i.e., 100% precision and recall) relations are used (⑦) vs. when mem-write extracts and writes relations to memory (④) – but not by much. It is worth noting that the recall of MW[DOCRED(Val.)] is 0.578 when assessed on the gold data from the validation set, i.e., 57.8% of the gold validation set triples are correctly extracted and stored by the memory-write model. This indicates that MEMLLM achieves good PPL results with memory-write extracted relations, despite their less-than-perfect coverage.<sup>8</sup> The underlying reason is that specific target entities might be involved in multiple relation triples. Our memory-read calls can handle several queries

<sup>8</sup>Apart from recall, we also measure the precision of the extracted relations and the impact of refinements on DOCRED. For a thorough analysis of how these refinements affect memory-write performance, please refer to Appendix F.

simultaneously (as shown in Figure 2b). Thus, if at least one of these queries matches a relation triple that contains the target entity and returns it as the memory result, the memory-read call will be successful. This contributes to the effectiveness of the language modeling process despite the limited recall.

**Scaling the Stored Relation Triples.** In a real-world scenario, the size of the memory will get large. As a result, query results will also get larger and the risk of unhelpful or even incorrect information being returned from the memory increases. To investigate the effect of the size of the memory, we compare our main experiment (①, using the full Wikipedia, 49.7M relations) with two ablations that use only Wikipedia abstracts (②, 17.9M relations) and only the DOCRED validation set (③). Table 1 shows that there is a relatively small negative effect of memory size: ③ (memory stripped down to the relations generated only from the validation set documents) is only slightly better than ① (full memory).

## 5 Conclusion

In this paper, we present MEMLLM, a novel approach to endowing an LLM with memory read and write capabilities through finetuning. MEMLLM’s memory is structured in a triple format to store information as relationships, offering interpretability, structure and scalability. We devise a memory API that enables the finetuned model to perform language modeling tasks interactively, leveraging the information stored in memory. The finetuning process trains the model to initiate memory-write calls, allowing it to extract and store relationships based on user input. Concurrently, for the language modeling task, the model learns to initiate memory-read calls during its decoding process: it generates queries and incorporates the memory’s responses to continue the language modeling (LM) task. Based on the API, we create a training dataset that can be easily used to finetune any standard LLM to endow it with an explicit memory. We show that MEMLLM improves overall language modeling capabilities, significantly enhancing performance on texts involving entities that make use of previously extracted knowledge. For future work, we intend to extend the applicability of our solution across diverse domains by incorporating additional types of relationships. Moreover, we aim to broaden our evaluations to other tasks,

including closed-book QA, open-domain summarization and temporal QA.

## References

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. [Self-RAG: Learning to retrieve, generate, and critique through self-reflection](#). *arXiv preprint arXiv:2310.11511*.
- Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv:2004.05150*.
- Steven Bird and Edward Loper. 2004. [NLTK: The natural language toolkit](#). In *Proceedings of the ACL Interactive Poster and Demonstration Sessions*, pages 214–217, Barcelona, Spain. Association for Computational Linguistics.
- Aydar Bulatov, Yuri Kuratov, and Mikhail Burtsev. 2022. [Recurrent memory transformer](#). In *Advances in Neural Information Processing Systems*.
- Mikhail S Burtsev, Yuri Kuratov, Anton Peganov, and Grigory V Sapunov. 2020. Memory transformer. *arXiv preprint arXiv:2006.11527*.
- Howard Chen, Ramakanth Pasunuru, Jason Weston, and Asli Celikyilmaz. 2023. Walking down the memory maze: Beyond context limit through interactive reading. *arXiv preprint arXiv:2310.05029*.
- Xin Cheng, Di Luo, Xiuying Chen, Lemao Liu, Dongyan Zhao, and Rui Yan. 2023. [Lift yourself up: Retrieval-augmented text generation with self-memory](#). In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. [Learning phrase representations using RNN encoder-decoder for statistical machine translation](#). In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar. Association for Computational Linguistics.
- Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi,

- David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2023. [Palm: Scaling language modeling with pathways](#). *Journal of Machine Learning Research*, 24(240):1–113.
- Nicola De Cao, Wilker Aziz, and Ivan Titov. 2021. [Editing factual knowledge in language models](#). In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6491–6506, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.
- Jia-Chen Gu, Hao-Xiang Xu, Jun-Yu Ma, Pan Lu, Zhen-Hua Ling, Kai-Wei Chang, and Nanyun Peng. 2024. Model editing can hurt general abilities of large language models. *arXiv preprint arXiv:2401.04700*.
- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. Realm: retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning*, ICML’20. JMLR.org.
- Zexue He, Leonid Karlinsky, Donghyun Kim, Julian McAuley, Dmitry Krotov, and Rogerio Feris. 2024. Camelot: Towards large language models with training-free consolidated associative memory. *arXiv preprint arXiv:2402.13449*.
- Sepp Hochreiter and Jürgen Schmidhuber. 1997. [Long short-term memory](#). *Neural Computation*, 9(8):1735–1780.
- Chenxu Hu, Jie Fu, Chenzhuang Du, Simian Luo, Junbo Zhao, and Hang Zhao. 2023. Chatdb: Augmenting llms with databases as their symbolic memory. *arXiv preprint arXiv:2306.03901*.
- Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. [LoRA: Low-rank adaptation of large language models](#). In *International Conference on Learning Representations*.
- Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2022. [Unsupervised dense information retrieval with contrastive learning](#). *Transactions on Machine Learning Research*.
- Joel Jang, Seonghyeon Ye, Changho Lee, Sohee Yang, Joongbo Shin, Janghoon Han, Gyeonghun Kim, and Minjoon Seo. 2022. [TemporalWiki: A lifelong benchmark for training and evaluating ever-evolving language models](#). In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 6237–6250, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.
- Samy Jelassi, David Brandfonbrener, Sham M Kakade, and Eran Malach. 2024. Repeat after me: Transformers are better than state space models at copying. *arXiv preprint arXiv:2402.01032*.
- Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.
- Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023a. Mistral 7b. *arXiv preprint arXiv:2310.06825*.
- Zhengbao Jiang, Frank Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023b. [Active retrieval augmented generation](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992, Singapore. Association for Computational Linguistics.
- Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. 2023. Large language models struggle to learn long-tail knowledge. In *Proceedings of the 40th International Conference on Machine Learning*, ICML’23. JMLR.org.
- Jungo Kasai, Keisuke Sakaguchi, yoichi takahashi, Ronan Le Bras, Akari Asai, Xinyan Velocity Yu, Dragomir Radev, Noah A. Smith, Yejin Choi, and Kentaro Inui. 2023. [Realtime QA: What’s the answer right now?](#) In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Diederik Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, San Diego, CA, USA.
- Mosh Levy, Alon Jacoby, and Yoav Goldberg. 2024. Same task, more tokens: the impact of input length on the reasoning performance of large language models. *arXiv:2402.14848*.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. [Retrieval-augmented generation for knowledge-intensive nlp tasks](#). In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.
- Xinnian Liang, Bing Wang, Hui Huang, Shuangzhi Wu, Peihao Wu, Lu Lu, Zejun Ma, and Zhoujun

- Li. 2023. Enhancing large language model with self-controlled memory framework. *arXiv preprint arXiv:2304.13343*.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the middle: How language models use long contexts. *arXiv:2307.03172*.
- Qi Liu, Dani Yogatama, and Phil Blunsom. 2022. [Relational memory-augmented language models](#). *Transactions of the Association for Computational Linguistics*, 10:555–572.
- Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4):824–836.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. [When not to trust language models: Investigating effectiveness of parametric and non-parametric memories](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9802–9822, Toronto, Canada. Association for Computational Linguistics.
- Pedro Henrique Martins, Zita Marinho, and Andre Martins. 2022.  [\$\infty\$ -former: Infinite memory transformer](#). In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5468–5485, Dublin, Ireland. Association for Computational Linguistics.
- Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. [On faithfulness and factuality in abstractive summarization](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1906–1919, Online. Association for Computational Linguistics.
- Mike Mintz, Steven Bills, Rion Snow, and Daniel Jurafsky. 2009. [Distant supervision for relation extraction without labeled data](#). In *Proceedings of the Joint Conference of the 47th Annual Meeting of the ACL and the 4th International Joint Conference on Natural Language Processing of the AFNLP*, pages 1003–1011, Suntec, Singapore. Association for Computational Linguistics.
- Eric Mitchell, Charles Lin, Antoine Bosselut, Christopher D Manning, and Chelsea Finn. 2022. [Memory-based model editing at scale](#). In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 15817–15831. PMLR.
- Ali Modarressi, Ayyoob Imani, Mohsen Fayyaz, and Hinrich Schütze. 2023. Ret-llm: Towards a general read-write memory for large language models. *arXiv preprint arXiv:2305.14322*.
- Charles Packer, Vivian Fang, Shishir G Patil, Kevin Lin, Sarah Wooders, and Joseph E Gonzalez. 2023. [Memgpt: Towards llms as operating systems](#). *arXiv preprint arXiv:2310.08560*.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulators of human behavior. In *In the 36th Annual ACM Symposium on User Interface Software and Technology (UIST ’23)*, UIST ’23, New York, NY, USA. Association for Computing Machinery.
- Adam Roberts, Colin Raffel, and Noam Shazeer. 2020. [How much knowledge can you pack into the parameters of a language model?](#) In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5418–5426, Online. Association for Computational Linguistics.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. [Toolformer: Language models can teach themselves to use tools](#). In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. 2023. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning, ICML’23*. JMLR.org.
- Anton Sinitin, Vsevolod Plokhotnyuk, Dmitry Pyrkov, Sergei Popov, and Artem Babenko. 2020. [Editable neural networks](#). In *International Conference on Learning Representations*.
- Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-attention with linear complexity. *arXiv:2006.04768*.
- Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. 2023a. [Augmenting language models with long-term memory](#). In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Yu Wang, Xiusi Chen, Jingbo Shang, and Julian McAuley. 2024. Memoryllm: Towards self-updatable large language models. *arXiv preprint arXiv:2402.04624*.
- Zekun Wang, Ge Zhang, Kexin Yang, Ning Shi, Wangchunshu Zhou, Shaochun Hao, Guangzheng Xiong, Yizhi Li, Mong Yuan Sim, Xiuying Chen, et al. 2023b. Interactive natural language processing. *arXiv preprint arXiv:2305.13246*.
- Qingyang Wu, Zhenzhong Lan, Kun Qian, Jing Gu, Alborz Geramifard, and Zhou Yu. 2022a. [Memformer: A memory-augmented transformer for sequence modeling](#). In *Findings of the Association for Computational Linguistics: AACL-IJCNLP 2022*, pages 308–318, Online only. Association for Computational Linguistics.

Yuhuai Wu, Markus Norman Rabe, DeLesley Hutchins, and Christian Szegedy. 2022b. [Memorizing transformers](#). In *International Conference on Learning Representations*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023a. [React: Synergizing reasoning and acting in language models](#). In *The Eleventh International Conference on Learning Representations*.

Yuan Yao, Deming Ye, Peng Li, Xu Han, Yankai Lin, Zhenghao Liu, Zhiyuan Liu, Lixin Huang, Jie Zhou, and Maosong Sun. 2019. [DocRED: A large-scale document-level relation extraction dataset](#). In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 764–777, Florence, Italy. Association for Computational Linguistics.

Yunzhi Yao, Peng Wang, Bozhong Tian, Siyuan Cheng, Zhoubo Li, Shumin Deng, Huajun Chen, and Ningyu Zhang. 2023b. [Editing large language models: Problems, methods, and opportunities](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10222–10240, Singapore. Association for Computational Linguistics.

Wenhao Yu, Dan Iter, Shuohang Wang, Yichong Xu, Mingxuan Ju, Soumya Sanyal, Chenguang Zhu, Michael Zeng, and Meng Jiang. 2023. [Generate rather than retrieve: Large language models are strong context generators](#). In *The Eleventh International Conference on Learning Representations*.

Wangchunshu Zhou, Yuchen Eleanor Jiang, Peng Cui, Tiannan Wang, Zhenxin Xiao, Yifan Hou, Ryan Cotterell, and Mrinmaya Sachan. 2023. Recurrentgpt: Interactive generation of (arbitrarily) long text. *arXiv preprint arXiv:2305.13304*.

## A Memory-write Decoding Method

While one might use RET-LLM with greedy decoding for memory writes, we suggest that the fine-tuned model may end the memory-write too early, before completely extracting all relations. Therefore, to ensure the model captures all relevant relations, we implement a late stopping strategy. In this approach, similar to greedy decoding, we consistently select the top-scoring token as the next token, unless it’s the closing token “}”. If the closing token scores highest, we note its position, calculate the average log probability score of the sequence up to that point, and proceed with the second highest scoring token—typically the “;” separator—resuming greedy decoding. By tracking the positions where the closing token was predicted, along with their corresponding logprob

scores, we maintain the generation process until there are no enhancements in the scores for  $K=5$  consecutive times. Subsequently, we halt the generation and select the position with the highest score as the cutoff point.

## B Filtering Ambiguous Queries

As we aim to assist the model with the stored memory content, having concise query results would facilitate reaching this objective. Getting precise outputs from the memory would require queries that are tailored in a way which lead to an exact match or related entities to the target entity. To reduce the chances of getting a vast and wide-range amount of outputs from the memory, we exclude queries that potentially leads to such results. In Table 2, we demonstrate query patterns that we intuitively assume based on the queried entity and the relation type that would lead to an ambiguous result. Therefore, we drop any query that would match with one of the mentioned patterns.

## C Anticipating Early Memory-read Calls

Ideally, memory-read calls should be initiated prior to generating a target entity. However, during the evaluation of language modeling on a text, the model may opt for an early memory-read call. This would deteriorate any perplexity-based evaluation since the gold text is predetermined. If the model invokes a call with some tokens remaining to decode before reaching the target tokens, the resulting perplexity for those additional tokens would be excessively high. This is because the model anticipates generating an entity immediately after the call.

To mitigate this and enhance decoding robustness, we duplicate each memory-read training example, creating two instances. The first copy is subjected solely to the training loss concerning  $y[x^{\text{MR}}]$ , representing the API up to the memory results. In the second duplicate, we adjust the API position to an earlier point within  $y[x^{\text{MR}}]$  by a random number of tokens, determined by a Poisson distribution ( $\lambda = 1$ ). Moreover, in this instance, the loss is only limited to the continuation up to the subsequent initial token,  $y^{\text{postMR}}$ . Note that the next API call would still start at the correct (unshifted) position. Therefore, with this workaround, the model would still be trained to predict the correct positions for memory-read calls while also being robust on early calls.

| Query (q)                       | Relation type ( $t^q$ )                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\langle *, t^q, e_o^q \rangle$ | country of citizenship, country, country of origin,<br>religion, place of birth, place of death, work location,<br>location, basin country, residence, location of formation,<br>publication date, production company, platform,<br>original language of work, applies to jurisdiction,<br>located in the administrative territorial entity,<br>headquarters location, inception,<br>employer, date of birth, date of death, educated at |
| $\langle e_s^q, t^q, * \rangle$ | contains administrative territorial entity                                                                                                                                                                                                                                                                                                                                                                                               |

Table 2: List of ambiguous queries subject to the filtering process.

## D Filtering Distant Supervision Relations

To increase the number of training examples, we also include examples from the distant supervision subset of DOCRED. Distant supervision (Mintz et al., 2009) assumes that a relation  $r$  exists between two entities ( $e_s, e_o$ ) in a text if the text includes both entities and the  $r = \langle e_s, t, e_o \rangle$  relation triple exists in a knowledge base. While this method is valuable for relation extraction, it may introduce noisy examples without any evidence of the relation in the text. This noise could adversely affect our training pipeline.

The experimental setup is as follows: We start with a partial document ( $S = \{s_1, s_2, \dots, s_i\}$ ) mentioning two entities ( $e_1, e_2$ ), with at least one of them present in the last sentence (i.e., the focus sentence),  $s_i$ . Our aim is to determine whether the potential relation  $r$  between  $e_1$  and  $e_2$  has any evidence in the last sentence.

To filter out negative examples, we use large language models (i.e., Mixtral). We design 8-shot in-context learning examples to detect if there is evidence of a relation in the focus sentence. Initially, we curate a test set to evaluate the performance of this filtering mechanism. We select 1000 examples from the human-annotated split of DOCRED as positive examples where the focus sentence is annotated as evidence. For negative examples, we choose 1000 examples where the focus sentence contains at least one entity but there is no evidence for the relation in the focus sentence.

For prompting, we apply three different strategies. In the first approach (baseline), we expect the LLM to answer with “Yes” or “No” to report if the focus sentence contains evidence. With the second approach (justification), we expect the LLM

| Filtering Approach | Prec.       | Rec.        | F1          | Acc.        |
|--------------------|-------------|-------------|-------------|-------------|
| Baseline           | 0.58        | 0.83        | 0.68        | 0.61        |
| Justification      | 0.56        | 0.82        | 0.66        | 0.59        |
| Reasoning          | <b>0.78</b> | <b>0.84</b> | <b>0.80</b> | <b>0.80</b> |

Table 3: Comparing performance of different prompting strategy for filtering distant supervision data. The reasoning approach similar to chain-of-thought prompting performs best among other strategies.

to provide justification after giving the answer. In the final approach (reasoning), we expect the LLM to generate a natural sentence representing the relation, then provide reasoning, and finally generate the answer with “Yes” or “No” in the last sentence, similar to chain-of-thought prompting.

We present the initial results in Table 3. The results suggest that the reasoning approach outperforms the other two approaches by a large margin. Also, it suggests that the filtering would lead to higher quality based on the high recall score, 0.84. We demonstrate the best-performing prompt in Figure 4.

After applying this method, we combine the filtered distant dataset with the human-annotated data. Due to the annotated data’s significantly smaller size compared to the distant split, we oversample the former by a factor of 10 before incorporating it into the training data.

## E Hyperparameters Details

We finetune MEMLLM, with a Mistral-7B-v0.1 model (Jiang et al., 2023a) using an Adam optimizer (Kingma and Ba, 2015), with the learning rate set to  $5 \times 10^{-5}$ , 2 epochs, and a batch size of 96. For LoRA specific parameters, we apply a dropout rate of 0.1, with a rank of 32 and an alpha

|                              | Rec.         | Prec.        | Prec.<br>(MB) | F1           |
|------------------------------|--------------|--------------|---------------|--------------|
| Distant.                     | 0.576        | 0.309        | 0.582         | 0.579        |
| + Filtering                  | 0.517        | <b>0.454</b> | <b>0.827</b>  | 0.636        |
| + 85% NoRel. Drop            | 0.544        | 0.424        | 0.799         | 0.647        |
| + 10xAnnot.                  | <b>0.578</b> | 0.425        | 0.797         | <b>0.670</b> |
| <i>Without late stopping</i> |              |              |               |              |
| Distant.                     | 0.541        | 0.325        | 0.587         | 0.563        |
| + All                        | 0.562        | <b>0.448</b> | <b>0.818</b>  | 0.666        |

Table 4: Memory-write performance comparison across different training data compositions. The reported F1 score is the harmonic mean of recall and model-based (MB) precision values. (All: Filtering + 85% NoRel. Drop + 10xAnnot.)

weight of 8.

For the memory retrieval hyperparameters (§3.1), we set  $\tau_e$  and  $\tau_t$  to 0.7 and  $\tau_r$  to 0.85.

## F Memory write Ablation Study

As we applied various preprocessing steps on DO-CRED to enhance the distant supervised set, in this section we analyze the impact of these design choices on the extracted relations. We evaluate the performance of memory writes in terms of precision and recall, as shown in Table 4. Instead of relying solely on exact matches between extracted relations and gold labels, we utilize Contriever-based embeddings and thresholds outlined in §3.1 to determine matches. This approach accommodates similar relations that retain information from the original relation, which can be usable during memory-reads. Additionally, due to DO-CRED’s validation set having a false negative issue where not all existing relationships within its documents are fully annotated, we measure a model-based precision. This metric leverages the filtering mechanism introduced in D to determine whether a relation truly exists within a sentence.

Based on the results of the filtering process, we observe an improvement in precision with a comparatively small decline in recall. This improvement stems from the reduction in the number of relations per sentence, resulting in fewer relations extracted by the trained model. To counteract this effect, we discard 85% of training examples without any relations to extract. Although this slightly reduces precision, it enhances recall and yields a better F1 score compared to previous results. Furthermore, inclusion of the oversampled human-annotated set further boosts both recall and the overall F1 score.

As mentioned in A, we apply a late stopping decoding method for memory writes. To observe the impact, we compare the outputs of the trained model with greedy decoding versus late stopping. The results indicate that while greedy decoding enhances overall precision by limiting the generation of additional relations, it adversely affects recall and the overall F1 score. However, it does impact the recall and the overall F1 score. Given that memory-reads can tolerate ambiguity to some extent (§4.1), in scenarios where F1 scores are closely matched, achieving higher recall with broader coverage is preferable.

### Reasoning Prompt - Distant Supervision Filtering

To determine whether the main sentence contains information about the given relation, both the main sentence and the context will be provided. The goal is to identify whether there is evidence of the relation in the main sentence, supported by the context. If there is no relation or the evidence exists solely in the context without requiring the main sentence, respond with No. Otherwise, respond with Yes. Provide reasoning to support your response.

**Context:**

**Main Sentence:** James Michael Osting ( born April 7 , 1977 ) is a former Major League Baseball pitcher .

**Relation:** ("Osting", "date of birth", "April 7 , 1977")

**Evidence:** The relation indicates that Osting was born on April 7, 1977. The main sentence explicitly mentions that Osting was born on April 7, 1977. The answer is Yes.

**Context:** Splashdown is a Hot Tuna album released in 1984 containing the tracks from a previously unreleased live acoustic performance that had been played on the short - lived radio station WQIV in the mid-1970s . During the recording , news of the Apollo - Soyuz mission returning to Earth after the first USA - USSR rendezvous in space reached the station , and the astronauts ' radio transmissions were played at the same time as Jorma and Jack continued with " Police Dog Blues " . The transmissions mixed with the song were preserved for this release as the last track of side 1 .

**Main Sentence:** The album was Hot Tuna 's first release on Relix Records , and one of the first Relix releases .

**Relation:** ("Hot Tuna", "country of origin", "USA")

**Evidence:** The relation indicates that the origin of Hot Tuna is the country of the United States. There is no evidence in the main sentence regarding the country of origin of Hot Tuna. The answer is No.

**Context:**

**Main Sentence:** The Chemung Canal Bank Building is located at 415 East Water Street in Elmira , Chemung County , New York , United States .

**Relation:** ("Chemung County", "capital", "Elmira")

**Evidence:** The relation indicates that Elmira is the capital of Chemung County. The main sentence only specifies the location of Elmira within Chemung County but does not mention Elmira as the capital of Chemung County. The answer is No.

**Context:** Carrie Lam Cheng Yuet - ngor , GBM , GBS ( ; born 13 May 1957 ) is the 4th and current Chief Executive of Hong Kong . Before that she was the Chief Secretary for Administration , the most senior rank of principal officials of Hong Kong , from 2012 to 2017 .

**Main Sentence:** After graduating from the University of Hong Kong , Lam joined the civil service in 1980 and served in various bureaux and departments .

**Relation:** ("Lam", "educated at", "University of Hong Kong")

**Evidence:** The relation indicates that Lam received education at the University of Hong Kong. The main sentence mentions that Carrie Lam Cheng Yuet-ngor graduated from the University of Hong Kong. The answer is Yes.

**Context:** Pacific Fair is a major shopping centre in Broadbeach Waters on the Gold Coast , Queensland , Australia . It was Queensland 's largest regional shopping centre until 2006 . Pacific Fair was developed by Hooker Retail Developments and opened in 1977 on what was swampland with 96 specialty stores and two anchor tenants . Since then , Pacific Fair has undergone numerous expansions and has grown to have more than 300 specialty stores and four anchor tenants . In January 2014 , work began on a major redevelopment project to meet the predicted regional growth on the Gold Coast . Prior to the redevelopment , the shopping centre had four main major stores including a four - level Myer , Kmart , Target , Coles and Toys ' R ' Us . Daimaru operated in the centre before its Australian withdrawal , albeit briefly .

**Main Sentence:** It also had a 12-screen Birch Carroll and Coyle Cinema ( re - opened as Event Cinemas in late 2015 ) .

**Relation:** ("Event Cinemas", "country", "Australia")

**Evidence:** The relation indicates that Event Cinemas is located in the country of Australia. The main sentence mentions that Event Cinemas is part of Pacific Fair which is located in Australia. The answer is Yes.

**Context:** Benjamin Winslow Harris ( November 10 , 1823 - February 7 , 1907 ) was a nineteenth - century politician , lawyer and judge from Massachusetts . He was the father of Robert Orr Harris . Born in East Bridgewater , Massachusetts , Harris pursued an academic course at Phillips Academy , Andover , graduating in 1847 . He graduated from Dane Law School of Harvard University in 1849 . He was admitted to the bar in Boston , Massachusetts in 1850 , commencing practice in East Bridgewater . He served in the Massachusetts Senate in 1857 , was a member of the Massachusetts House of Representatives in 1858 , was district attorney for the southeastern district of Massachusetts from 1858 to 1866 and was collector of internal revenue for the second district of Massachusetts from 1866 to 1873 . Harris was elected a Republican to the United States House of Representatives in 1872 , serving from 1873 to 1883 , not being a candidate for renomination in 1882 . There , he served as chairman of the Committee on Naval Affairs from 1881 to 1883 . Afterwards , he resumed practicing law in East Bridgewater , Massachusetts and was judge of probate for Plymouth County , Massachusetts from 1887 to 1906 .

**Main Sentence:** Harris died in East Bridgewater on February 7 , 1907 and was interred in Central Cemetery in East Bridgewater .

**Relation:** ("Benjamin Winslow Harris", "place of birth", "East Bridgewater")

**Evidence:** The relation indicates that Benjamin Winslow Harris was born in East Bridgewater. The main sentence lacks information about Benjamin Winslow Harris's place of birth. The evidence for East Bridgewater as his birthplace is exclusively found in the context, not in the main sentence. The answer is No.

**Context:** Greatest Hats is the first compilation album by the Canadian new wave / synthpop group Men Without Hats , released in 1996 .

**Main Sentence:** A slightly modified version of the album was released in the US in 1996 , entitled Collection .

**Relation:** ("Collection", "performer", "Men Without Hats")

**Evidence:** The relation indicates that Men Without Hats is the performer of the Collection album. The main sentence says that Men Without Hats released slightly modified version of the Greatest Hats album which is the album Collection. The answer is Yes.

**Context:** Aaron Hobart ( June 26 , 1787 - September 19 , 1858 ) was a U.S. Representative from Massachusetts . Born in Abington , Massachusetts , Hobart pursued classical studies and graduated from Brown University in 1805 . He studied law , was admitted to the bar and commenced practice in Abington . He served as member of the Massachusetts House of Representatives and served in the Massachusetts State Senate . Hobart was elected as a Democratic - Republican to the Sixteenth Congress to fill the vacancy caused by the resignation of Zabdiel Sampson . He was reelected as a Democratic - Republican to the Seventeenth Congress , elected as an Adams - Clay Republican to the Eighteenth Congress , and reelected as an Adams candidate to the Nineteenth Congress , and served from November 24 , 1820 , to March 3 , 1827 . He declined to be a candidate for renomination in 1826 .

**Main Sentence:** He then served as an Executive councilor 1827 - 1831 and served as probate judge 1843 - 1858 .

**Relation:** ("Aaron Hobart", "date of death", "1858")

**Evidence:** The relation indicates that Aaron Hobart passed away in the year 1858. The main sentence does not contain information about the given relation. The evidence of Aaron Hobart's date of death in 1858 is solely present in the context and is not mentioned in the provided main sentence. The answer is No.

**Context:** [[context]]

**Main Sentence:** [[focus\_sentence]]

**Relation:** ("[[entity1]]", "[[relation]]", "[[entity2]]")

**Evidence:**

Figure 4: The prompt for the distant supervision dataset filtering. This prompt includes the natural representation of the relation, the reasoning, and the final answer.